

1 Rue de la Chebarde, 63170,
Aubière, France

Projet d'Ingénieur ISIMA 3^{ème} année

Modélisation des systèmes de décision multi-agents centralisés en temps réel basés sur les préférences à l'aide de l'apprentissage par renforcement et de la théorie des jeux

Filière : Optimisation et Intelligence Artificielle

Presenté par : **Célio Lucas LEMES DE MEDEIROS**

Responsable ISIMA : Viet Hung NGUYEN

ISIMA - Campus des Cézeaux. 1 rue de la Chebarde. TSA 60125. 63178 Aubière CEDEX

Résumé

Ce travail est le résultat d'un projet réalisé en 3^{ème} année d'Ingénierie Informatique à l'Institut Supérieur d'Informatique, de Modélisation et de leurs Applications (ISIMA).

Il s'agit d'une étude de recherche dans le domaine de l'apprentissage par renforcement (RL), portant sur la conception d'un environnement d'apprentissage dédié à la problématique de la recharge d'hydrogène. À travers des demandes de service et une flotte de véhicules disponibles pour répondre à cette demande, l'environnement a été construit en abstraisant les éléments réels impliqués dans ce problème, tout en intégrant un algorithme d'optimisation permettant d'établir le couplage entre véhicules et demandes à chaque instant.

Afin de favoriser l'apprentissage, un algorithme d'apprentissage par renforcement profond a été mis en œuvre dans le but de maximiser l'efficacité du système et de déterminer les actions optimales que les agents (véhicules) doivent entreprendre à chaque instant. Enfin, les résultats obtenus sont présentés à partir de l'environnement modélisé et implémenté, ainsi que l'évaluation des performances de l'algorithme d'apprentissage par renforcement appliqué, en le comparant à différentes politiques pour en analyser l'efficacité.

Mots-clés : Problématique de la Recharge d'Hydrogène, Apprentissage par Renforcement Multi-Agent, Listes de Préférences, Gale-Shapley, Gradient de Politique Déterministe Profond, Réseaux Neuronaux, Python.

Abstract

This work is the result of a project carried out during the 3rd year of Computer Engineering at the Institut Supérieur d'Informatique, de Modélisation et de leurs Applications (ISIMA).

It is a research study in the field of reinforcement learning (RL), focusing on the design of a learning environment dedicated to the hydrogen refueling problem. Through service requests and a fleet of vehicles available to meet this demand, the environment was built by abstracting the real-world elements involved in this problem while integrating an optimization algorithm to establish the pairing between vehicles and requests at each moment.

To enable learning, a deep reinforcement learning algorithm was implemented with the objective of maximizing system efficiency and determining the optimal actions that agents (vehicles) should take at each step. Finally, the obtained results are presented based on the modeled and implemented environment, along with an evaluation of the performance of the applied reinforcement learning algorithm, compared against different policies to assess its effectiveness.

Keywords : Hydrogen Refueling Problem, Multi-Agent Reinforcement Learning, Preference Lists, Gale-Shapley, Deep Deterministic Policy Gradient, Neural Networks, Python.

Remerciements

Je tiens à exprimer mes sincères remerciements à M. Viet Hung NGUYEN, qui a proposé et supervisé ce travail en tant que tuteur, ainsi qu'au M. Rafael COLARES BORGES, qui a également apporté un soutien précieux tout au long du projet de ma 3^{ème} année. Leur direction et leur aide pour résoudre les questions qui se sont posées ont été déterminantes pour la réalisation et la finalisation de ce projet.

Table des matières

Résumé	i
Abstract	i
Remerciements	ii
Table des figures	v
Table des tableaux	vi
1 Présentation du Sujet	2
1.1 Contexte du Sujet	3
2 Modélisation Mathématique	5
2.1 Ensemble des Véhicules (V)	5
2.2 Ensemble des Commandes (C)	5
2.3 Matrices de Pondération	6
2.3.1 Matrice de Pondération des Commandes (W^C)	6
2.3.2 Matrice de Pondération des Véhicules (W^V)	6
2.4 Espace d'Observation	7
2.5 Fonctions de Score	7
2.5.1 Fonction de Score pour les Véhicules (S)	8
2.5.2 Fonction de Score pour les Commandes (S')	9
2.5.3 Comparaison et Relation entre les Fonctions de Score	9
2.5.4 Normalisation et Considérations Générales	10
3 Description Formelle du Fonctionnement de l'Environnement	11
3.1 Initialisation des Matrices de Pondération	11
3.2 Fonctions de Score et Listes de Préférence	11
3.3 Algorithme de Gale-Shapley	12
3.4 Récompenses et Mise à Jour des États	12
3.5 Mise à Jour des Positions et des Conditions de Trafic	12
3.6 Consommation d'Hydrogène et Temps de Travail	13
3.7 Indisponibilité Temporaire des Véhicules	13
3.8 Génération Dynamique des Commandes	14
3.9 États Terminaux et Politique d'Inaction	14
4 Description de l'Algorithme d'Apprentissage par Renforcement	15
5 Résultats et Discussion	20
5.1 Convergence de l'algorithme	20
5.2 Résultats des tests	21
5.2.1 Scénario avec 3 véhicules	21
5.2.2 Scénario avec 5 véhicules	22
5.2.3 Scénario avec 7 véhicules	23
5.3 Comparaison avec les politiques alternatives	24

5.4	Analyse de l'évolutivité	27
5.5	Discussion des résultats	28
5.5.1	Comparaison des performances par scénario	28
6	Conclusion	29
	Références	vii

Table des figures

1	Interaction entre les Véhicules et les Commandes avec l'algorithme d'affectation	4
2	Diagramme d'entraînement et d'exécution pour l'algorithme d'apprentissage.	15
3	Architecture du réseau neuronal pour le Critic	17
4	Architecture du réseau neuronal pour l'Actor	18
5	Convergence de l'entraînement pour 3 véhicules.	20
6	Convergence de l'entraînement pour 5 véhicules.	21
7	Convergence de l'entraînement pour 7 véhicules.	21
8	Récompense moyenne pour le test de l'algorithme avec 3 véhicules.	22
9	Nombre de commandes au fil des étapes pour 3 véhicules.	22
10	Récompense moyenne pour le test de l'algorithme avec 5 véhicules.	23
11	Nombre de commandes au fil des étapes pour 5 véhicules.	23
12	Récompense moyenne pour le test de l'algorithme avec 7 véhicules.	24
13	Nombre de commandes au fil des étapes pour 7 véhicules.	24
14	Récompenses moyennes obtenues pour des actions aléatoires avec 3 véhicules.	25
15	Récompenses moyennes obtenues en valorisant uniquement le prix et l'urgence avec 3 véhicules.	25
16	Récompenses moyennes obtenues pour des actions aléatoires avec 5 véhicules.	26
17	Récompenses moyennes obtenues en valorisant uniquement le prix et l'urgence avec 5 véhicules.	26
18	Récompenses moyennes obtenues pour des actions aléatoires avec 7 véhicules.	27
19	Récompenses moyennes obtenues en valorisant uniquement le prix et l'urgence avec 7 véhicules.	27

Liste des tableaux

1	Attributs des Commandes (C) et des Véhicules (V)	3
2	Statistiques descriptives pour le scénario avec 3 véhicules.	22
3	Statistiques descriptives pour le scénario avec 5 véhicules.	23
4	Statistiques descriptives pour le scénario avec 7 véhicules.	24
5	Temps d'entraînement pour différents nombres d'agents.	27
6	Comparaison des performances pour le scénario avec 3 véhicules.	28
7	Comparaison des performances pour le scénario avec 5 véhicules.	28
8	Comparaison des performances pour le scénario avec 7 véhicules.	28

Introduction

Ce document présente une étude de recherche sur l'utilisation de la théorie des jeux et de l'apprentissage par renforcement (RL) dans le cadre d'un service en ligne à la demande pour des systèmes multi-agents. Les travaux menés dans [1] et [2] constituent la base de la formalisation mathématique présentée dans ce rapport.

Dans le scénario spécifique abordé par ces études, la modélisation repose sur un environnement d'apprentissage par renforcement défini sur une grille de 100×100 . Le système fonctionne à travers deux ensembles principaux : les Véhicules V et les Commandes C . Chaque ensemble est caractérisé par des attributs spécifiques, et des poids sont attribués à ces attributs en fonction de leur pertinence pour les éléments de l'ensemble opposé. Ces interactions seront détaillées tout au long du rapport.

L'environnement est conçu pour prendre en compte les préférences et générer des appariements entre les Commandes et les Véhicules en utilisant l'algorithme d'optimisation de Gale-Shapley, tel qu'introduit et mis en œuvre dans [2] - [4]. Pour chaque paire résultante (v, c) , où $v \in V$ et $c \in C$, les Véhicules reçoivent des récompenses en fonction de la valeur du score s attribué à la Commande qui leur a été assignée par l'algorithme d'affectation, selon leur liste de préférences l .

En implémentant un algorithme d'apprentissage par renforcement adapté à ce problème, l'objectif est de permettre aux Véhicules, considérés comme des agents, d'ajuster les poids qu'ils attribuent aux attributs d'une Commande c . Cet ajustement vise à optimiser la liste de préférences ordonnée l , construite dans un ordre décroissant en fonction des scores s_c calculés pour chaque Commande. Par conséquent, l'algorithme d'appariement retournera les paires (v, c) les mieux classées en tenant compte des listes de préférences de chaque Véhicule l_v et de chaque Commande l_c , améliorant ainsi l'efficacité globale du système.

1 Présentation du Sujet

L'apprentissage par renforcement, tel que décrit par [5], est un cadre dans lequel un agent apprend à identifier les actions les plus appropriées à entreprendre dans un contexte donné, en s'appuyant sur les récompenses qu'il reçoit en retour de ses actions. Un aspect fondamental de ce processus est que les actions d'un agent n'ont pas seulement un effet immédiat, mais influencent également les états futurs, les décisions ultérieures et, en fin de compte, les récompenses accumulées. Cette dynamique d'exploration par essais et erreurs, couplée à un mécanisme de retour d'information sous forme de récompenses, constitue le cœur même de l'apprentissage par renforcement.

Dans ce cadre, l'apprentissage par renforcement permet d'entraîner des agents à maximiser leurs récompenses cumulées (de long terme) dans un environnement caractérisé par ses états, ses actions et ses objectifs. Sutton et Barto [5] identifient quatre éléments clés qui forment la base des systèmes d'apprentissage par renforcement :

1. **Politique** : La politique définit le comportement de l'agent en établissant une correspondance entre un état s à un instant donné t et une action a . Les politiques peuvent être déterministes, attribuant une action unique à chaque état, ou stochastiques, spécifiant une distribution de probabilité sur un ensemble d'actions possibles. Une politique efficace permet à l'agent d'optimiser son interaction avec l'environnement afin d'atteindre un objectif donné.
2. **Signal de récompense** : Le signal de récompense est une rétroaction numérique fournie par l'environnement à chaque instant t . Il indique si les actions de l'agent ont été bénéfiques ou non. Une faible récompense suggère que la politique de l'agent pourrait nécessiter un ajustement, l'objectif étant de maximiser les récompenses au fil des itérations. L'apprentissage par renforcement repose sur l'accumulation de ces signaux afin d'orienter l'agent vers une stratégie optimale.
3. **Fonction de valeur** : Si le signal de récompense fournit une évaluation instantanée de la qualité d'une action, la fonction de valeur estime les bénéfices attendus à long terme d'un état s . Cette fonction prédit la récompense cumulative qu'un agent peut espérer en considérant les états futurs probables et les récompenses associées. Une bonne approximation de la fonction de valeur permet de prendre des décisions plus stratégiques en anticipant les conséquences d'une action sur le long terme.
4. **Modèle** : Le modèle simule la dynamique de l'environnement. Il permet de prédire l'état suivant s' ainsi que la récompense r à partir de l'état actuel s et de l'action a . L'utilisation d'un modèle peut faciliter la prise de décision en permettant à l'agent d'évaluer plusieurs scénarios futurs sans nécessairement interagir directement avec l'environnement.

En s'appuyant sur ces concepts fondamentaux, l'objectif de ce travail est de concevoir un cadre d'apprentissage par renforcement et de mettre en œuvre un algorithme permettant aux agents d'acquérir un modèle de leur environnement. Grâce à cette approche, les agents seront en mesure d'adapter leur comportement de manière optimale afin d'atteindre une performance maximale dans les tâches qui leur sont assignées.

1.1 Contexte du Sujet

Dans les scénarios du monde réel, de nombreux services nécessitent la résolution d'un problème d'affectation entre les utilisateurs (ou clients) et les prestataires de services. Des exemples incluent les services de taxi, où des individus dispersés demandent des trajets et où des véhicules sont envoyés pour répondre à la demande ; les services de livraison, où des coursiers reçoivent des demandes pour récupérer des repas dans des restaurants et les livrer aux clients ; et les services d'urgence, tels que les pompiers, la police et les ambulances, où des véhicules sont mobilisés pour répondre aux demandes d'assistance à travers une ville. Ce travail aborde une situation similaire dans le contexte spécifique de recharge d'hydrogène, où des clients demandent un service de recharge et où des Véhicules, agissant comme stations mobiles, sont disponibles pour effectuer les Commandes.

Ce scénario implique deux entités principales : **Commandes** (C) et **Véhicules** (V), chacune caractérisée par des attributs spécifiques. Pour assurer la faisabilité de l'implémentation, certains attributs du monde réel ont été omis, et seul un sous-ensemble a été considéré, comme résumé dans le Tableau 1 :

Élément	Attributs
Commandes (C)	Position
	Prix
	Durée du service
	Condition du trafic
	Urgence
Véhicules (V)	Position
	Quantité d'hydrogène disponible
	Temps de travail restant
	Qualité du service
	Condition du trafic

TABLE 1 – Attributs des Commandes (C) et des Véhicules (V)

Dans ce cadre, chaque Commande $c \in C$ établit une liste de préférences sur l'ensemble des Véhicules V en fonction de leurs attributs, tandis que chaque Véhicule $v \in V$ établit de la même manière une liste de préférences sur les Commandes C . Les préférences sont calculées à l'aide d'un modèle de somme pondérée, où chaque entité attribue des poids w_i aux attributs de l'autre entité.

Pour déterminer l'affectation entre les Commandes et les Véhicules, l'algorithme de Gale-Shapley est utilisé. Cet algorithme traite les listes de préférences des deux ensembles et génère de manière itérative une affectation stable, garantissant qu'aucune paire Commande-Véhicule ne préférerait être associée dans les possibilités autrement que dans

leur affectation actuelle. La Figure 1 illustre l'interaction entre les Commandes et les Véhicules, médiée par l'algorithme d'affectation.

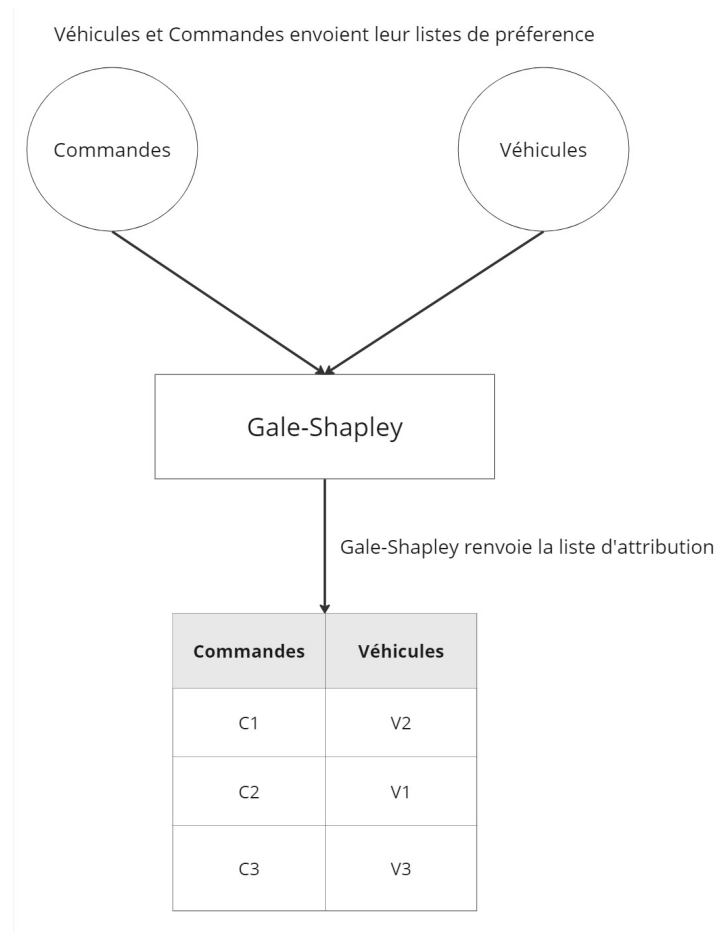


FIGURE 1 – Interaction entre les Véhicules et les Commandes avec l'algorithme d'affectation

Un aspect clé de ce processus est la priorité accordée aux listes de préférences lors de l'affectation. En suivant l'approche décrite dans [2], l'algorithme privilégie les préférences des Commandes, considérées comme les clients dans l'analogie du monde réel. Cela signifie que l'objectif principal est de satisfaire, autant que possible, la première préférence de chaque Commande. Cependant, des conflits surgissent lorsque plusieurs Commandes priorisent le même Véhicule, et les préférences des Véhicules doivent également être prises en compte pour résoudre ces conflits.

Cette dynamique introduit la nécessité d'un apprentissage. Les Véhicules doivent adapter leurs listes de préférences en ajustant les poids w_i^v attribués aux attributs des Commandes. L'objectif est de maximiser leurs récompenses tout en respectant la priorisation des Commandes dans le processus d'affectation. Grâce à l'apprentissage par renforcement, les Véhicules apprennent à optimiser leurs scores de préférence s_c^v , garantissant ainsi que les premières préférences des Commandes sont satisfaites tout en améliorant leurs propres résultats au sein du système.

2 Modélisation Mathématique

Dans cette section, nous définissons les ensembles et les attributs associés nécessaires à la modélisation du problème. Ces ensembles sont utilisés pour représenter les entités principales du système étudié, à savoir les véhicules et les commandes.

2.1 Ensemble des Véhicules (V)

Soit V l'ensemble des véhicules. Chaque véhicule $v \in V$ est caractérisé par un tuple d'attributs défini comme suit :

$$A \subseteq D_1 \times D_2 \times D_3 \times D_4 \times D_5$$

où les domaines D_i sont définis par :

- $a^1 \in D_1$, avec $D_1 = \{(x, y) \mid x, y \in \{0, 1, 2, \dots, 99\}\}$, représentant la position du véhicule dans un environnement 2D discret de taille 100×100 .
- $a^2 \in D_2$, avec $D_2 = \{h \mid h \in \{80, 130, 180, \dots, 5000\}\}$, représentant la quantité d'hydrogène disponible dans le réservoir du véhicule, variant de 80 litres à sa capacité maximale de 5000 litres.
- $a^3 \in D_3$, avec $D_3 = \{t \mid t \in \{5, 15, 25, \dots, 480\}\}$, représentant le temps opérationnel restant du véhicule, exprimé en minutes, pour une journée de travail complète de 8 heures (480 minutes).
- $a^4 \in D_4$, avec $D_4 = \{q \mid q \in \{1, 2, 3, 4, 5\}\}$, représentant la qualité de service fournie par le véhicule, évaluée sur une échelle discrète de 1 à 5.
- $a^5 \in D_5$, avec $D_5 = \{b \mid b \in \{0, 1\}\}$, représentant les conditions de trafic rencontrées par le véhicule, où $b = 0$ indique un trafic fluide et $b = 1$ un trafic dense. Ces conditions dépendent de la localisation (x, y) du véhicule.

Chaque attribut a^i est généré à partir d'une distribution uniforme $U(a, b)$, où a et b sont les bornes inférieure et supérieure du domaine D_i .

2.2 Ensemble des Commandes (C)

Soit C l'ensemble des commandes. Chaque commande $c \in C$ est caractérisée par un tuple d'attributs défini comme suit :

$$B \subseteq D_1 \times D_2 \times D_3 \times D_4 \times D_5$$

où les domaines D_i sont définis par :

- $b^1 \in D_1$, avec $D_1 = \{(x, y) \mid x, y \in \{1, 2, 3, \dots, 100\}\}$, représentant la position de la commande dans le même environnement 2D discret que celui utilisé pour les véhicules.
- $b^2 \in D_2$, avec $D_2 = \{p \mid p \in \{20, 25, 30, \dots, 500\}\}$, représentant le prix de la commande, exprimé en unités monétaires, et variant en fonction du type de client servi.
- $b^3 \in D_3$, avec $D_3 = \{d \mid d \in \{5, 6, 7, \dots, 20\}\}$, représentant la durée de la commande, exprimée en minutes.
- $b^4 \in D_4$, avec $D_4 = \{b \mid b \in \{0, 1\}\}$, représentant les conditions de trafic à la localisation de la commande, selon les mêmes règles que pour les véhicules.
- $b^5 \in D_5$, avec $D_5 = \{u \mid u \in \{0, 1\}\}$, représentant l'urgence de la commande, où $u = 1$ indique une commande urgente et $u = 0$ une commande standard.

Comme pour les véhicules, chaque attribut b^i est généré à partir d'une distribution uniforme $U(a, b)$, où a et b sont les bornes inférieure et supérieure du domaine D_i .

2.3 Matrices de Pondération

Le système intègre des matrices de pondération pour attribuer des niveaux d'importance aux différents attributs des véhicules et des commandes. Ces matrices permettent de quantifier l'influence de chaque attribut dans les décisions du système.

2.3.1 Matrice de Pondération des Commandes (W^C)

Soit $W^C \in [0, 1]^{m \times 4}$ la matrice de pondération des commandes, où m est le nombre de commandes et 4 représente le nombre d'attributs pertinents. Chaque élément w_i^c de W^C est défini comme suit :

$$w_i^c \in [0, 1]^4 \quad \text{avec} \quad w_i^c \sim U(0, 1),$$

où w_i^c est un vecteur de poids associé à la commande c . Ces poids sont générés selon une distribution uniforme $U(0, 1)$ et indiquent l'importance relative de chaque attribut des véhicules pour la commande c . Chaque ligne de W^C correspond à un vecteur de poids pour une commande spécifique, tandis que les colonnes représentent les attributs des véhicules à évaluer.

2.3.2 Matrice de Pondération des Véhicules (W^V)

De manière analogue, soit $W^V \in [0, 1]^{n \times 4}$ la matrice de pondération des véhicules, où n est le nombre de véhicules. Chaque élément w_i^v de W^V est défini comme suit :

$$w_i^v \in [0, 1]^4 \quad \text{avec} \quad w_i^v \sim U(0, 1),$$

où w_i^v est un vecteur de poids associé au véhicule v . Ces poids sont optimisés par un algorithme d'apprentissage par renforcement, qui ajuste leurs valeurs en fonction de l'état s du véhicule et des performances du système.

2.4 Espace d'Observation

L'espace d'observation O représente l'environnement tel qu'il est perçu par chaque véhicule v à un état s donné. Il est défini comme un tuple $o_v \in O$, où :

$$O \subseteq \prod_{i=1}^{13} D_i.$$

Les composantes de o_v sont définies comme suit :

- $o^1 \in D_1$: position actuelle du véhicule v .
- $o^2 \in D_2$: distance entre le véhicule v et chaque commande c .
- $o^3 \in D_3$: quantité d'hydrogène disponible dans le réservoir du véhicule.
- $o^4 \in D_4$: temps opérationnel restant du véhicule.
- $o^5 \in D_5$: qualité de service fournie par le véhicule.
- $o^6 \in D_6$: indicateur binaire indiquant si le véhicule est actuellement en train de servir une commande.
- $o^7 \in D_7$: indicateur binaire pour la condition de trafic à la position actuelle du véhicule.
- $o^8 \in D_8$: prix des commandes disponibles.
- $o^9 \in D_9$: durée des commandes.
- $o^{10} \in D_{10}$: valeurs binaires de condition de trafic à la position de chaque commande.
- $o^{11} \in D_{11}$: urgence de chaque commande (0 pour non urgente, 1 pour urgente).
- $o^{12} \in D_{12}$: nombre total de commandes dans l'environnement.
- $o^{13} \in D_{13}$: poids choisis par chaque commande pour les attributs des véhicules.

Ces informations sont mises à jour à chaque instant t et permettent au véhicule de prendre des décisions éclairées en fonction de l'état actuel de l'environnement. L'espace d'observation est essentiel pour l'apprentissage par renforcement, car il fournit les données nécessaires pour évaluer les actions et ajuster les politiques de décision.

2.5 Fonctions de Score

Les fonctions de score sont utilisées pour évaluer l'adéquation entre les véhicules et les commandes dans le système. Leur objectif principal est de générer des listes de préférence pour l'algorithme d'appariement stable, en l'occurrence l'algorithme de Gale-Shapley, décrit en [4]. Deux fonctions distinctes sont définies : une pour évaluer les véhicules par

rapport aux commandes (S) et une autre pour évaluer les commandes par rapport aux véhicules (S'). Ces fonctions permettent de prendre des décisions optimales en fonction des objectifs du système, tels que :

- La maximisation du revenu,
- La minimisation du temps d'attente pour la réalisation d'un service,
- La priorisation des commandes urgentes.

2.5.1 Fonction de Score pour les Véhicules (S)

La fonction de score $S(c, w^v, v_{x,y})$ évalue l'adéquation d'un véhicule v pour une commande c en fonction des attributs de la commande et des poids associés au véhicule. Cette fonction est définie par :

$$S(c, w^v, v_{x,y}) = w_1 \cdot p - w_2 \cdot D(c, v) - w_3 \cdot d + w_4 \cdot u, \quad (1)$$

où :

- p : prix de la commande c ,
- $c = (c_x, c_y)$: position de la commande dans l'environnement 2D,
- $v = (v_x, v_y)$: position actuelle du véhicule dans l'environnement 2D,
- d : durée de la commande, exprimée en minutes,
- u : urgence de la commande, avec $u \in \{0, 1\}$ (0 pour non urgente, 1 pour urgente),
- w_1, w_2, w_3, w_4 : poids du vecteur w^v , représentant l'importance relative de chaque attribut dans le calcul du score,
- $D(c, v)$: distance euclidienne entre la commande c et le véhicule v , calculée par :

$$D(c, v) = \sqrt{(c_x - v_x)^2 + (c_y - v_y)^2}. \quad (2)$$

Pénalisation due au trafic dense Si la commande c se trouve dans une zone de trafic dense, la distance $D(c, v)$ est pénalisée par un facteur de 1,5. La formule ajustée devient alors :

$$S(c, w^v, v_{x,y}) = w_1 \cdot p - w_2 \cdot \mathbf{1.5} \cdot D(c, v) - w_3 \cdot d + w_4 \cdot u. \quad (3)$$

Interprétation des termes La fonction de score $S(c, w^v, v_{x,y})$ peut être interprétée comme suit :

- Le terme $w_1 \cdot p$ favorise les commandes avec un prix élevé, ce qui maximise le revenu.
- Le terme $-w_2 \cdot D(c, v)$ pénalise les commandes éloignées du véhicule, réduisant ainsi le temps et les coûts de déplacement.
- Le terme $-w_3 \cdot d$ pénalise les commandes de longue durée, permettant au véhicule de servir plus de commandes dans un temps donné.
- Le terme $+w_4 \cdot u$ favorise les commandes urgentes, assurant une réponse rapide aux demandes critiques.

2.5.2 Fonction de Score pour les Commandes (S')

La fonction de score $S'(v, w^c, c_{x,y})$ évalue l'adéquation d'une commande c pour un véhicule v en fonction des attributs du véhicule et des poids associés à la commande. Cette fonction est définie par :

$$S'(v, w^c, c_{x,y}) = w_1 \cdot h - w_2 \cdot D(c, v) + w_3 \cdot t + w_4 \cdot q, \quad (4)$$

où :

- h : quantité d'hydrogène disponible dans le véhicule v ,
- t : temps opérationnel restant du véhicule v ,
- q : qualité de service du véhicule v ,
- w_1, w_2, w_3, w_4 : poids du vecteur w^c , représentant l'importance relative de chaque attribut dans le calcul du score,
- $D(c, v)$: distance euclidienne entre la commande c et le véhicule v , calculée par l'équation (2).

Pénalisation due au trafic dense Comme pour les véhicules, les commandes prennent en compte les conditions de trafic à leur position. Si le véhicule v se trouve dans une zone de trafic dense, la distance $D(c, v)$ est pénalisée par un facteur de 1,5. La formule ajustée devient alors :

$$S'(v, w^c, c_{x,y}) = w_1 \cdot h - w_2 \cdot \mathbf{1.5} \cdot D(c, v) + w_3 \cdot t + w_4 \cdot q. \quad (5)$$

Interprétation des termes La fonction de score $S'(v, w^c, c_{x,y})$ peut être interprétée comme suit :

- Le terme $w_1 \cdot h$ favorise les véhicules avec une plus grande quantité d'hydrogène disponible, réduisant les risques de panne sèche.
- Le terme $-w_2 \cdot D(c, v)$ pénalise les véhicules éloignés de la commande, réduisant ainsi le temps d'attente pour le service.
- Le terme $+w_3 \cdot t$ favorise les véhicules avec plus de temps opérationnel restant, minimisant les risques de non-atteinte des objectifs.
- Le terme $+w_4 \cdot q$ favorise les véhicules avec un indice de qualité de service plus élevé.

2.5.3 Comparaison et Relation entre les Fonctions de Score

Les fonctions S et S' sont complémentaires :

- S évalue les véhicules en fonction des commandes, en se concentrant sur des attributs tels que le prix, la distance, la durée et l'urgence.

- S' évalue les commandes en fonction des véhicules, en considérant des attributs tels que l'hydrogène disponible, la distance, le temps opérationnel restant et la qualité de service.

Ensemble, ces fonctions permettent une prise de décision équilibrée, en tenant compte à la fois des besoins des commandes et des véhicules.

2.5.4 Normalisation et Considérations Générales

Pour garantir une évaluation équitable, tous les attributs sont normalisés dans l'intervalle $[0, 1]$ avant le calcul des scores. Cette normalisation permet de comparer des grandeurs hétérogènes sur une même échelle. De plus, les pénalisations dues au trafic dense sont intégrées pour refléter les conditions réelles de l'environnement, assurant ainsi des décisions plus réalistes et optimisées.

3 Description Formelle du Fonctionnement de l'Environnement

3.1 Initialisation des Matrices de Pondération

Étant donné les attributs précédemment définis pour chaque véhicule $v \in V$ et chaque commande $c \in C$, les matrices de pondération associées sont initialisées pour permettre les interactions dans l'environnement. La matrice de pondération pour les véhicules $\mathbf{W}^V$ et pour les commandes $\mathbf{W}^C$ est générée aléatoirement selon une distribution uniforme $U(0, 1) \in \mathbb{R}$. L'algorithme d'apprentissage vise à faire converger la matrice de pondération des véhicules $\mathbf{W}^V$, tandis que la matrice $\mathbf{W}^C$ est modifiée à chaque interaction lors de la génération de nouvelles commandes de manière aléatoire.

3.2 Fonctions de Score et Listes de Préférence

La fonction $S(c, w^v, v_{x,y})$ est appelée pour chaque paire (v, c) , renvoyant une valeur réelle qui représente le score de chaque véhicule par rapport à chaque commande. De même, la fonction $S'(v, w^c, c_{x,y})$ est utilisée pour évaluer les commandes par rapport aux véhicules.

Ces valeurs de score sont utilisées pour générer des listes de préférences individuelles pour chaque véhicule et chaque commande. La liste de préférences d'un véhicule v est donnée par :

$$\mathcal{L}_v = \{c_1, c_2, \dots, c_m\}$$

où c_m est la m -ième commande dans le classement des préférences pour v . Les commandes $c \in C$ sont ordonnées de manière non croissante selon la valeur du score donné par le vecteur de scores s^v défini par l'équation (1). De même, chaque commande c aura sa liste de préférences $\mathcal{L}_c$ ordonnée de manière non croissante, définie à partir du vecteur de scores s^c retourné par la fonction de score pour les commandes, comme défini dans (4), indiquant la préférence de chaque commande par rapport à chaque véhicule. Ainsi, nous avons :

$$\mathcal{L}_c = \{v_1, v_2, \dots, v_n\}$$

où v_n est le n -ième véhicule dans la liste de préférences d'une commande c .

3.3 Algorithme de Gale-Shapley

L'algorithme de Gale-Shapley est utilisé pour assigner chaque véhicule v à une commande c , générant un ensemble de paires $\{(v_1, c_1), (v_2, c_2), \dots, (v_n, c_m)\}$ selon les listes de préférences générées. Cet algorithme peut être formalisé comme un problème d'optimisation où la fonction objectif est de maximiser le nombre total de paires assignées en tenant compte des préférences.

3.4 Récompenses et Mise à Jour des États

La récompense r reçue par chaque véhicule v est donnée par le score retourné par la fonction de score (1) de la commande c assignée à partir de la liste de préférences $\mathcal{L}_v$. Formellement, nous avons :

$$r(v) = s_i^v$$

où i est la commande assignée au véhicule v dans la liste de préférences $\mathcal{L}_v$. Plus la valeur du score retournée pour la commande c est élevée, meilleure sera la récompense.

Ce processus est répété jusqu'à ce que l'algorithme d'apprentissage converge, ajustant les poids $\mathbf{W}^V$ pour maximiser la récompense reçue par chaque véhicule en fonction de son état actuel s .

3.5 Mise à Jour des Positions et des Conditions de Trafic

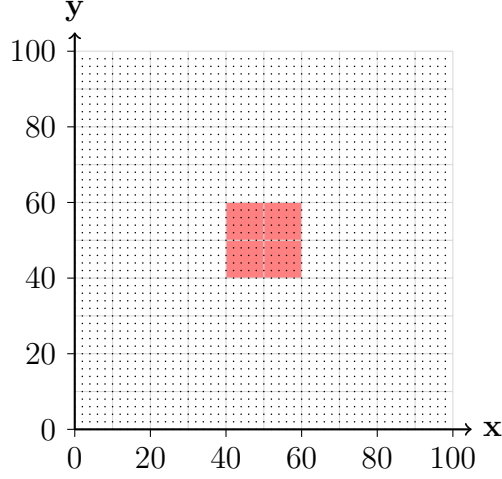
Après chaque pas de temps, chaque véhicule v se déplace vers la position de sa commande assignée c . Cette mise à jour de position peut être représentée mathématiquement comme suit :

$$v_x^{t+1} = c_x^t, \quad v_y^{t+1} = c_y^t$$

où t représente le pas de temps actuel, et $t + 1$ représente le pas de temps suivant. De plus, la condition de trafic est mise à jour en fonction de la nouvelle position du véhicule, comme suit :

$$b(v^{t+1}) = \begin{cases} 1 & \text{si } 40 \leq v_x^{t+1} \leq 60 \text{ et } 40 \leq v_y^{t+1} \leq 60 \\ 0 & \text{sinon} \end{cases}$$

Ainsi, la condition de trafic difficile est définie dans la région centrale de la grille de l'environnement, entre les coordonnées (40, 40) et (60, 60), comme illustré dans la figure ci-dessous :



Lorsque les commandes sont initialisées dans cette région, leur attribut b de condition de trafic est également mis à jour à 1, indiquant une région de déplacement plus difficile. Cela est utilisé pour calculer le score en utilisant les équations (3) et (5).

3.6 Consommation d'Hydrogène et Temps de Travail

Le niveau d'hydrogène disponible pour chaque véhicule est ajusté en fonction de son taux de consommation. Le niveau d'hydrogène mis à jour v_h^{t+1} est donné par :

$$v_h^{t+1} = \max(v_h^t - (v_d^t \cdot 80), 0)$$

où :

- v_h^t représente le niveau d'hydrogène du véhicule v au pas de temps t ,
- v_d^t représente la durée du service effectué par le véhicule v au pas de temps t ,
- 80 est le taux de consommation d'hydrogène par minute.

De même, le temps de travail restant v_t^{t+1} est mis à jour comme suit :

$$v_t^{t+1} = \max(v_t^t - v_d^t, 0)$$

Cette formulation garantit que le niveau d'hydrogène et le temps de travail restant ne deviennent pas négatifs, reflétant ainsi les contraintes physiques du système.

3.7 Indisponibilité Temporaire des Véhicules

Pendant l'exécution de l'environnement, les véhicules peuvent rester inactifs pendant un pas de temps en raison de la non-finalisation d'un service. Pour les cas où la commande traitée a une durée supérieure à 10 minutes, le véhicule assigné sera indisponible pour traiter de nouvelles commandes au pas de temps suivant. Ainsi, le véhicule ne pourra agir à nouveau qu'au pas de temps $t + 2$. Formellement, cela peut être exprimé comme suit :

$$\text{Affecté}(v^{t+1}) = \begin{cases} 1 & \text{si } v_d^t > 10 \text{ minutes} \\ 0 & \text{sinon} \end{cases}$$

où v_d^t représente la durée de la commande traitée par le véhicule v au pas de temps t . Cette règle garantit que les véhicules ne soient pas surchargés et qu'ils aient suffisamment de temps pour terminer les services de longue durée avant de pouvoir accepter de nouvelles commandes.

3.8 Génération Dynamique des Commandes

À chaque pas de temps t , les commandes sont traitées et de nouvelles sont générées pour que les véhicules continuent à agir. Le nombre de commandes à chaque pas de temps suit une distribution de Poisson, où λ est égal au nombre de véhicules n . Formellement :

$$m^t \sim \text{Poisson}(\lambda = n)$$

Cela entraîne une variation positive et négative du nombre de commandes par rapport aux véhicules à différents instants. Les poids de la matrice $\mathbf{W}^C$ sont également mis à jour aléatoirement à chaque pas de temps pour les nouvelles commandes générées.

3.9 États Terminaux et Politique d'Inaction

Un véhicule entre dans un état terminal lorsque son niveau d'hydrogène atteint 0 ou que son temps de travail restant est épuisé. Dans ce cas, la récompense $r(v)$ est automatiquement fixée à -5 , signalant une mauvaise décision. Formellement :

$$r(v) = \begin{cases} s_i^v & \text{si } v \text{ est assigné à une commande } i \\ -5 & \text{si } v \text{ entre dans un état terminal} \\ 0 & \text{si } \sum_{j=1}^4 w_j^v < 1 \text{ (inaction)} \end{cases}$$

Cette politique permet aux véhicules d'apprendre à identifier les commandes les plus opportunes en fonction de leurs caractéristiques, évitant ainsi les états terminaux et les récompenses négatives.

4 Description de l'Algorithme d'Apprentissage par Renforcement

Pour sélectionner l'algorithme d'apprentissage par renforcement appliqué dans ce travail, nous nous sommes référés à l'article [6], qui fournit un aperçu des méthodologies clés dans ce domaine. Cette étude démontre expérimentalement que le MADDPG (Multi-Agent Deep Deterministic Policy Gradient) offre des performances supérieures en termes de convergence et de stabilité dans divers scénarios.

Compte tenu de la modélisation de l'environnement, où les actions appartiennent à un espace continu dans l'intervalle $[0, 1]$, les algorithmes traditionnels tels que le DQN (Deep Q-Network) sont inefficaces. Cette inefficacité découle de la nécessité de calculer une distribution de probabilité pour chaque action dans l'espace d'état afin de déterminer une politique optimale π , comme discuté dans [5].

L'algorithme MADDPG est basé sur l'approche Actor-Critic, courante dans les méthodes de gradient de politique. Au lieu d'estimer la distribution de probabilité complète sur les actions, il apprend directement les paramètres de la distribution de la politique [5]. Plus précisément :

- L'**Actor** représente la politique de chaque agent et détermine les actions à entreprendre.
- Le **Critic** évalue la fonction de valeur-action en fonction des observations et des actions conjointes de tous les agents.

Ce concept est illustré dans le diagramme de [6], montré ci-dessous :

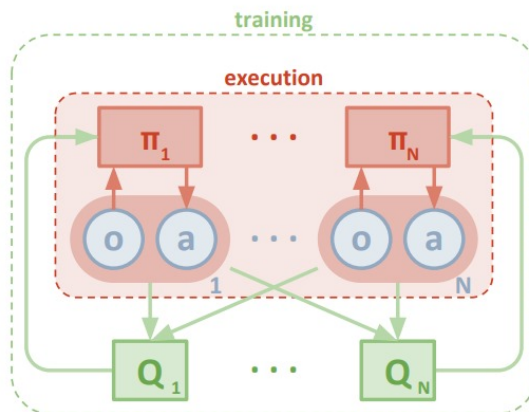


FIGURE 2 – Diagramme d'entraînement et d'exécution pour l'algorithme d'apprentissage.

Dans le diagramme, l'algorithme fonctionne avec une exécution décentralisée et un entraînement centralisé. Chaque agent observe indépendamment son état et sélectionne

des actions en fonction de sa politique. Cependant, pendant l'entraînement, la valeur Q est centralisée, incorporant les états et les actions de tous les agents. Cela permet à chaque agent de considérer l'environnement collectif à des fins d'apprentissage, ce qui entraîne une meilleure coopération et stabilité.

Grâce à ce cadre d'exécution décentralisée et d'entraînement centralisé, le MADDPG est adapté à la fois aux environnements multi-agents compétitifs et coopératifs. Dans cette étude, l'environnement est coopératif, car chaque véhicule vise à remplir la commande souhaitée en fonction des conditions établies pour les véhicules et les commandes, sans concurrence directe avec les autres.

L'algorithme complet, tel que présenté dans [6], est détaillé ci-dessous :

Algorithm 1 Multi-Agent Deep Deterministic Policy Gradient pour N agents

```

1: for épisode = 1 à  $M$  do
2:   Initialiser un processus aléatoire  $\mathcal{N}$  pour l'exploration des actions
3:   Recevoir l'état initial  $x$ 
4:   for  $t = 1$  à la longueur maximale de l'épisode do
5:     Pour chaque agent  $i$ , sélectionner l'action  $a_i = \mu_{\theta_i}(o_i) + \mathcal{N}_t$  basée sur la politique actuelle et l'exploration
6:     Exécuter les actions  $a = (a_1, \dots, a_N)$  et observer la récompense  $r$  et le nouvel état  $x'$ 
7:     Stocker  $(x, a, r, x')$  dans le replay buffer  $D$ 
8:     Mettre à jour l'état :  $x \leftarrow x'$ 
9:     for chaque agent  $i = 1$  à  $N$  do
10:      Échantillonner un minibatch aléatoire de  $S$  échantillons  $(x^j, a^j, r^j, x'^j)$  de  $D$ 
11:      Définir  $y^j = r_i^j + \gamma Q_{\mu_i}(x'^j, a^1, \dots, a^N)$ , où  $a'^k = \mu'_k(o_k^j)$ 
12:      Mettre à jour le critic en minimisant la perte :

```

$$L(\theta_i) = \frac{1}{S} \sum_j \left(y^j - Q_{\mu_i}(x^j, a_1^j, \dots, a_N^j) \right)^2$$

```

13:   Mettre à jour l'actor en utilisant le gradient de politique échantillonné :

```

$$\nabla_{\theta_i} J \approx \frac{1}{S} \sum_j \nabla_{\theta_i} \mu_i(o_i^j) \nabla_{a_i} Q_{\mu_i}(x^j, a_1^j, \dots, a_i, \dots, a_N^j) \Big|_{a_i = \mu_i(o_i^j)}$$

```

14:   end for
15:   Mettre à jour les paramètres du réseau cible pour chaque agent  $i$  :

```

$$\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$$

```

16:   end for
17: end for

```

Pour **Algorithme 1**, le nombre d'épisodes M et la longueur de l'épisode t (appelés étapes) doivent être définis. Analogiquement, les épisodes représentent des journées de travail complètes, tandis que les étapes totales dans un épisode représentent la séquence de décisions prises tout au long d'une seule journée.

Initialement, les actions sont prises aléatoirement en fonction d'une probabilité epsilon ϵ , qui décroît progressivement au fil du temps en utilisant un multiplicateur d . Ce mécanisme assure un équilibre entre l'exploration de l'espace d'action et l'exploitation des politiques apprises. Pour chaque action prise par un agent i , un tuple (x, a, r, x', d) est stocké dans la mémoire de replay. Dans ce tuple :

- x : Représente l'état de l'observation o avant l'action.
- a : Correspond à l'action entreprise par l'agent.
- r : Désigne la récompense reçue en retour de l'action.
- x' : Représente le nouvel état de l'observation o' après l'action.
- d : Indique si l'agent est entré dans un état terminal, comme décrit dans la section précédente.

L'état de l'agent est ensuite mis à jour pour refléter la transition.

Chaque agent maintient deux réseaux de neurones : un pour l'estimation de la valeur Q (**critic**) et un autre pour sa politique (**actor**). L'entraînement de ces réseaux est effectué de manière asynchrone. Une fois que la mémoire de replay D accumule un nombre suffisant de transitions S , un minibatch aléatoire d'expériences est échantillonné pour mettre à jour les réseaux. Cette méthode garantit que l'apprentissage n'est pas biaisé par des expériences temporellement corrélées, améliorant ainsi la stabilité.

Dans cette étude, après avoir réalisé l'hyperparamétrisation pour l'entraînement de l'agent d'apprentissage, un batch size de 512 a été défini pour l'entraînement, ainsi qu'une configuration des réseaux avec 2 couches cachées de 256 neurones chacune. Les réseaux pour l'entraînement du Critic et de l'Actor sont illustrés dans les figures ci-dessous :

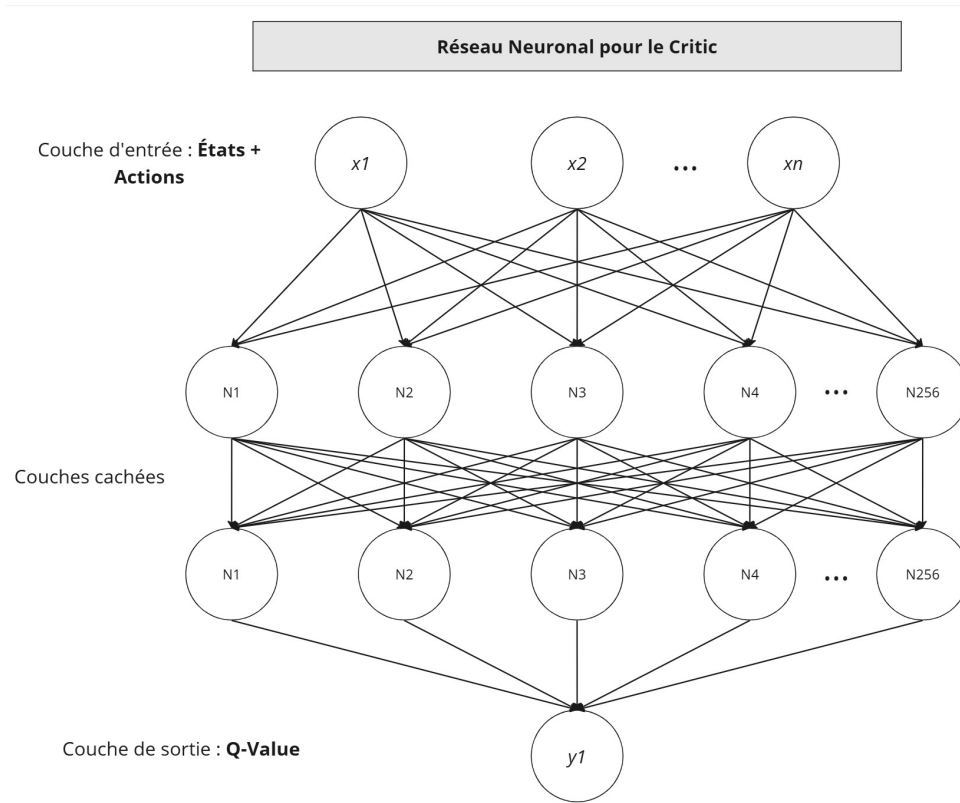


FIGURE 3 – Architecture du réseau neuronal pour le Critic

Où n représente le nombre d'agents dans l'environnement.

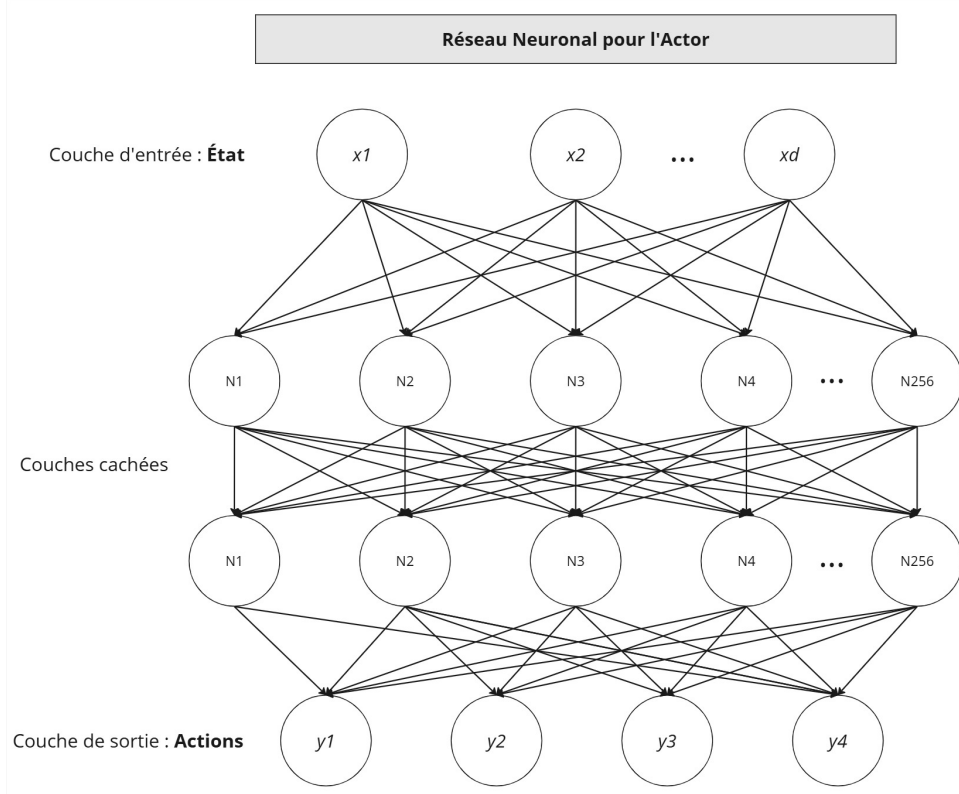


FIGURE 4 – Architecture du réseau neuronal pour l'Actor

Où d représente la dimension de l'état pour l'environnement, comme présenté dans la section de modélisation mathématique.

La valeur cible de la fonction Q est calculée en utilisant le critic, qui évalue les états actuels et les actions conjointes des N agents. Ces actions sont déterminées par le réseau de l'actor. Le réseau du critic est mis à jour en minimisant la fonction de perte $L(\theta_i)$, définie comme :

$$L(\theta_i) = \frac{1}{S} \sum_j (y^j - Q_{\mu_i}(x^j, a_1^j, \dots, a_N^j))^2,$$

où y^j est la valeur Q cible calculée comme :

$$y^j = r_i^j + \gamma Q_{\mu'_i}(x'^j, a'^1, \dots, a'^N),$$

et $a'^k = \mu'_k(\sigma_k^j)$. Cette formulation intègre les récompenses futures actualisées avec un facteur γ , permettant aux agents d'optimiser pour des objectifs à long terme.

Le réseau de l'actor est mis à jour en utilisant la méthode du gradient de politique. Plus précisément, l'actor apprend en ajustant ses paramètres dans la direction qui augmente la valeur Q . La mise à jour est donnée par :

$$\nabla_{\theta_i} J \approx \frac{1}{S} \sum_j \nabla_{\theta_i} \mu_i(\sigma_i^j) \nabla_{a_i} Q_{\mu_i}(x^j, a_1^j, \dots, a_i, \dots, a_N^j) \Big|_{a_i = \mu_i(\sigma_i^j)}.$$

Cette étape garantit que l'actor affine sa politique pour sélectionner des actions qui maxi-

misent le retour attendu.

Enfin, les paramètres des réseaux cibles (θ'_i) sont mis à jour en utilisant un mécanisme de mise à jour douce :

$$\theta'_i \leftarrow \tau\theta_i + (1 - \tau)\theta'_i,$$

où τ est le coefficient de mise à jour. Ce processus stabilise l'apprentissage en propageant lentement les mises à jour des réseaux principaux aux réseaux cibles, réduisant les oscillations pendant l'entraînement.

Avec les tuples (o_i, a_i, r_i, o'_i, d) , stockés en mémoire comme les expériences collectives des agents, les données nécessaires sont fournies pour entraîner itérativement les réseaux de l'actor et du critic. Cela permet aux agents d'optimiser leurs politiques et de s'adapter efficacement à l'environnement.

Pour la réalisation de l'entraînement, 15000 épisodes de 20 étapes chacun ont été définis, avec un décroissement de 0.01 de l'epsilon ϵ . Ce décroissement réduit progressivement les chances de prise de décision aléatoire, favorisant ainsi les décisions guidées par l'algorithme d'apprentissage. De plus, si tous les n agents entrent dans un état terminal, l'épisode en cours est terminé et le suivant est immédiatement commencé.

5 Résultats et Discussion

Cette section présente les résultats obtenus à partir de l'application de l'algorithme d'apprentissage par renforcement dans différents scénarios impliquant 3, 5 et 7 véhicules. Les performances de l'algorithme sont comparées à deux politiques alternatives : une prise de décision aléatoire et une maximisation des termes positifs (prix et urgence) de l'équation de score. Les résultats montrent que l'algorithme proposé surpasse les deux politiques en termes de récompenses moyennes et de nombre maximal d'étapes par épisode.

5.1 Convergence de l'algorithme

Pour évaluer l'efficacité de l'algorithme, celui-ci a été entraîné en variant le nombre d'agents à 3, 5 et 7 véhicules. La Figure 5 montre la convergence de l'algorithme pour 3 véhicules, avec une augmentation progressive des récompenses au fil des épisodes. On observe une tendance similaire pour les scénarios avec 5 et 7 véhicules, illustrés respectivement dans les Figures 6 et 7.

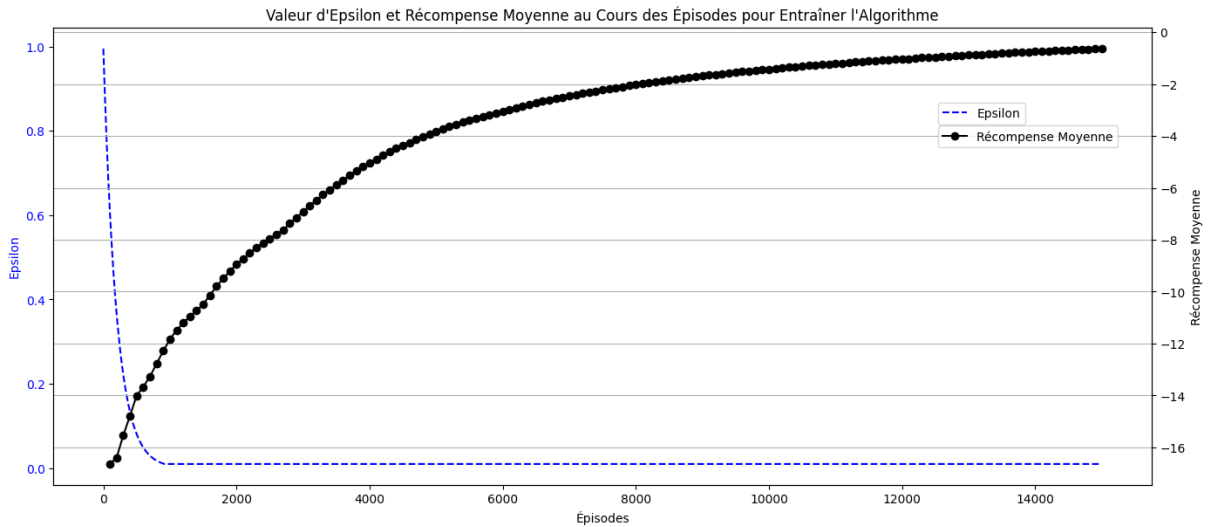


FIGURE 5 – Convergence de l'entraînement pour 3 véhicules.

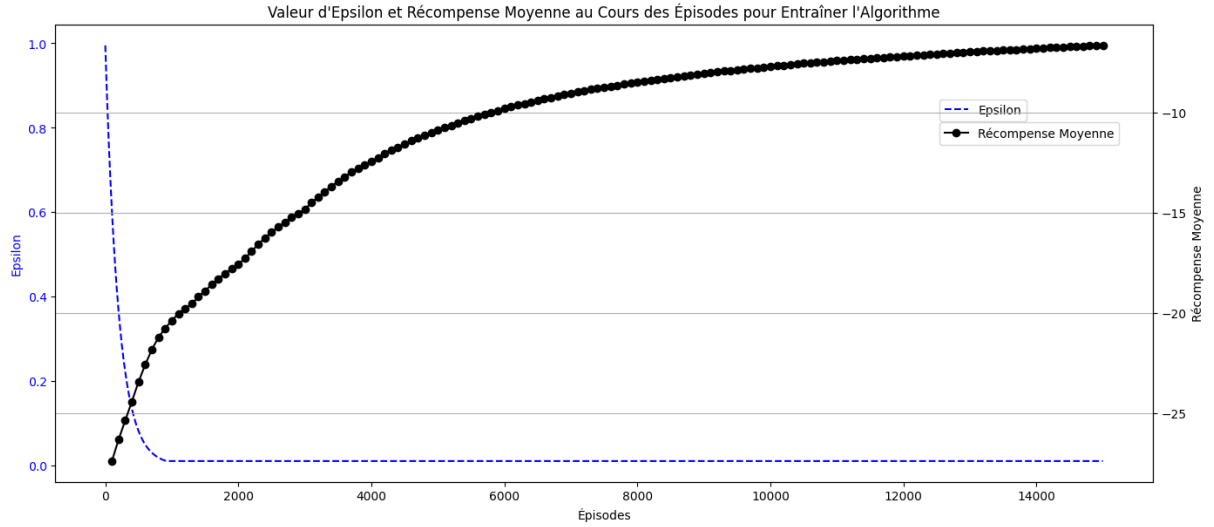


FIGURE 6 – Convergence de l'entraînement pour 5 véhicules.

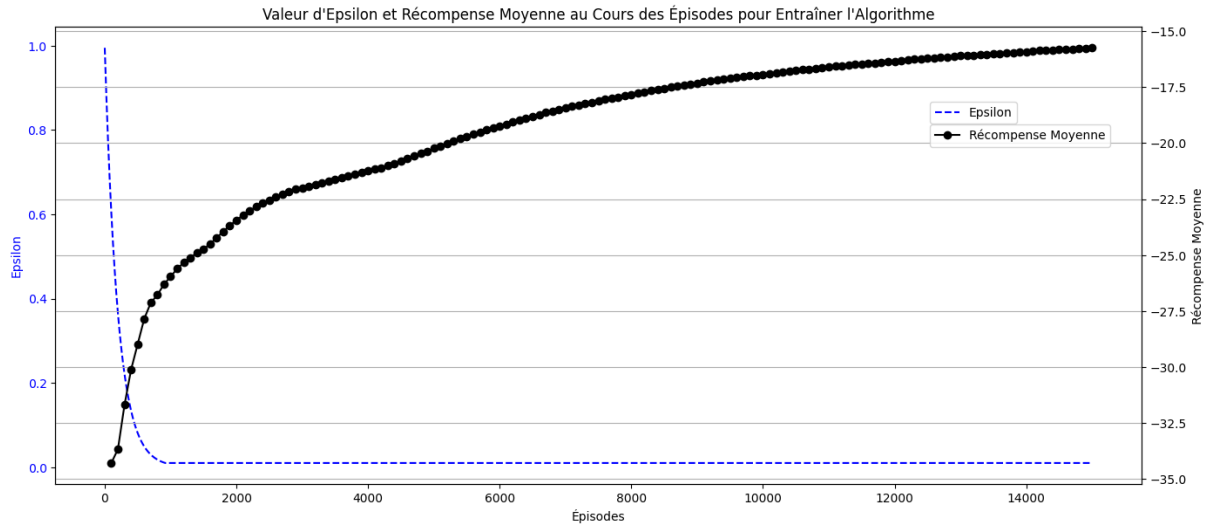


FIGURE 7 – Convergence de l'entraînement pour 7 véhicules.

5.2 Résultats des tests

Les résultats des tests pour chaque scénario sont présentés ci-dessous. Les récompenses moyennes et le nombre d'étapes sont analysés pour évaluer la performance de l'algorithme.

5.2.1 Scénario avec 3 véhicules

Pour le scénario avec 3 véhicules, les résultats des tests sont présentés dans la Figure 8. Les statistiques descriptives du nombre d'étapes et des récompenses moyennes sont les suivantes :

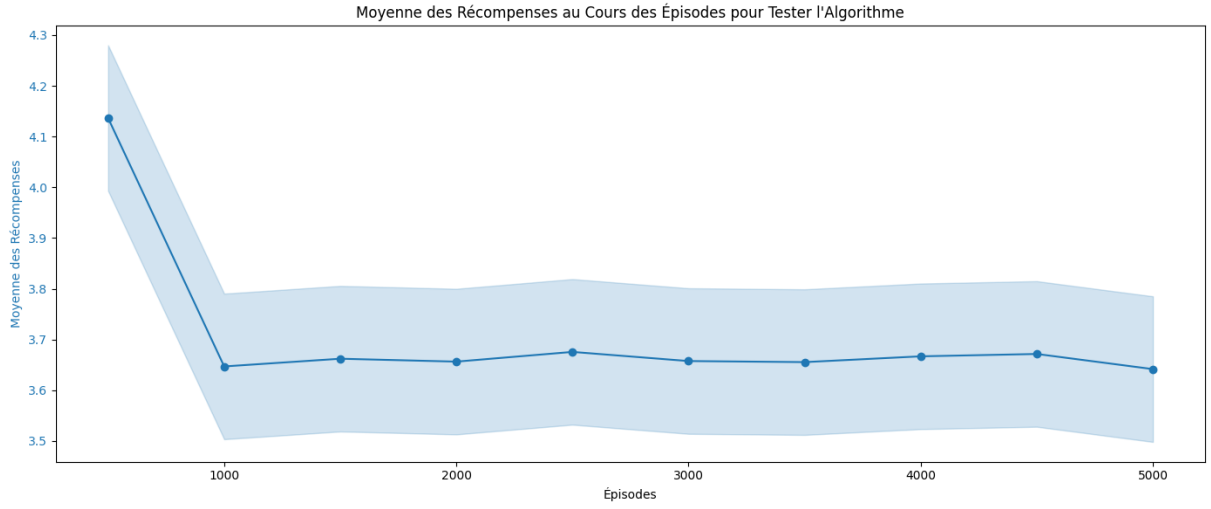


FIGURE 8 – Récompense moyenne pour le test de l'algorithme avec 3 véhicules.

Métrique	Mean	Std	Min	25%	50%	Max
Nombre d'étapes	20.0	0.0	20.0	20.0	20.0	20.0
Récompense moyenne	3.706961	0.151284	3.641543	3.655590	3.659738	4.136508

TABLE 2 – Statistiques descriptives pour le scénario avec 3 véhicules.

Pour illustrer la variation des commandes dans ce scénario, la Figure 9 montre leur évolution à chaque étape au cours des 5 premiers épisodes :

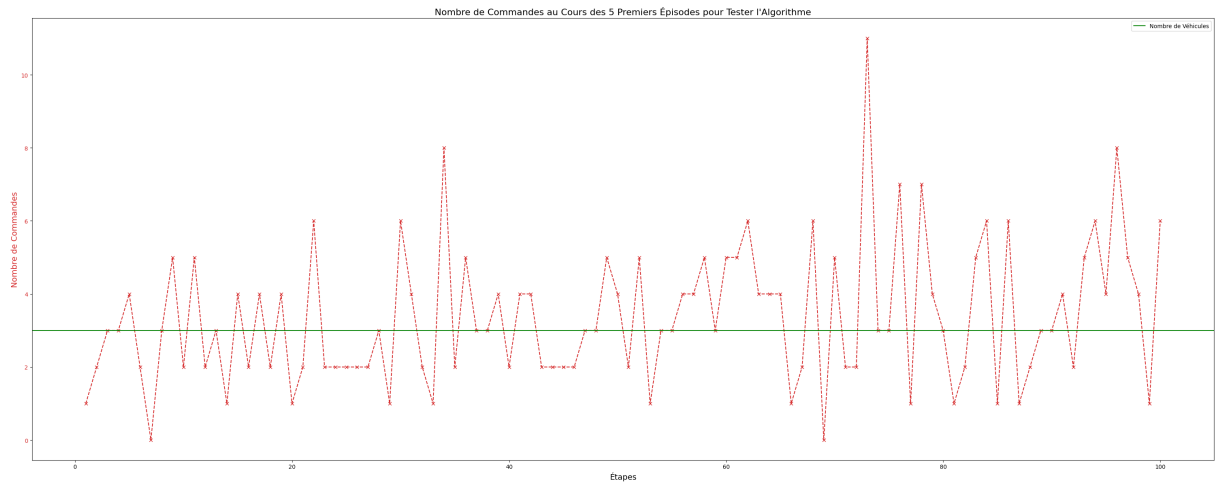


FIGURE 9 – Nombre de commandes au fil des étapes pour 3 véhicules.

5.2.2 Scénario avec 5 véhicules

Pour le scénario avec 5 véhicules, les résultats des tests sont présentés dans la Figure 10. Les statistiques descriptives du nombre d'étapes et des récompenses moyennes sont les suivantes :

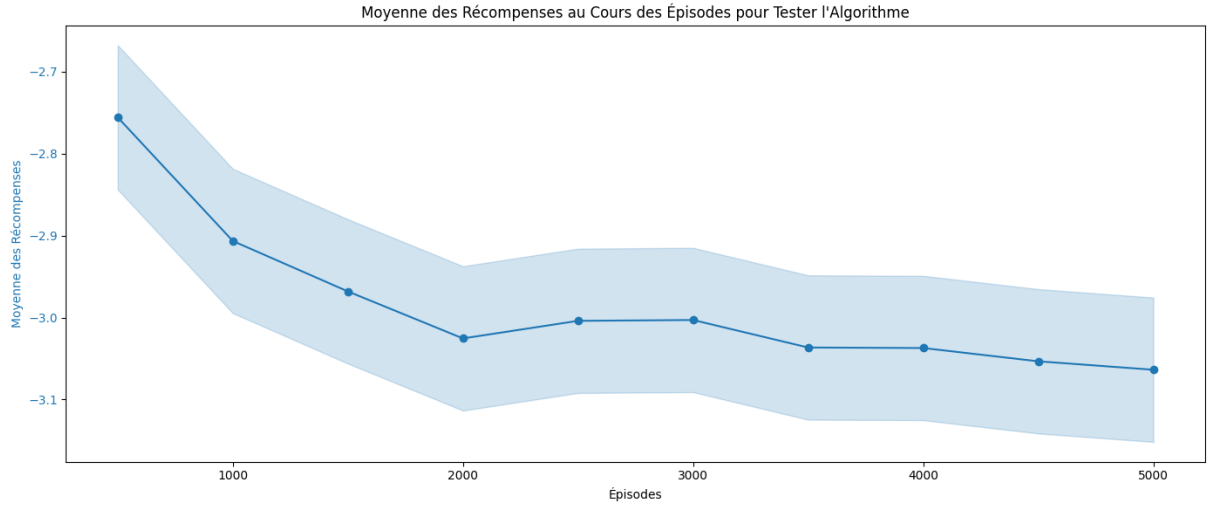


FIGURE 10 – Récompense moyenne pour le test de l'algorithme avec 5 véhicules.

Métrique	Mean	Std	Min	25%	50%	Max
Nombre d'étapes	20.0	0.0	20.0	20.0	20.0	20.0
Récompense moyenne	-2.985293	0.092818	-3.063602	-3.036938	-3.014647	-2.755708

TABLE 3 – Statistiques descriptives pour le scénario avec 5 véhicules.

La variation des commandes au cours des 5 premiers épisodes est illustrée dans la Figure 11 :

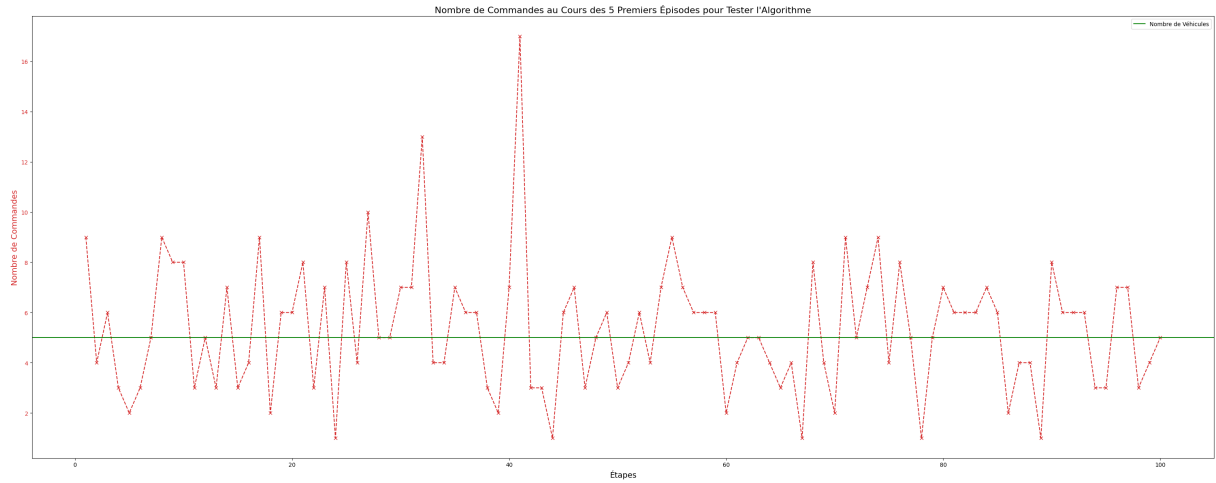


FIGURE 11 – Nombre de commandes au fil des étapes pour 5 véhicules.

5.2.3 Scénario avec 7 véhicules

Pour le scénario avec 7 véhicules, les résultats des tests sont présentés dans la Figure 12. Les statistiques descriptives du nombre d'étapes et des récompenses moyennes sont les suivantes :

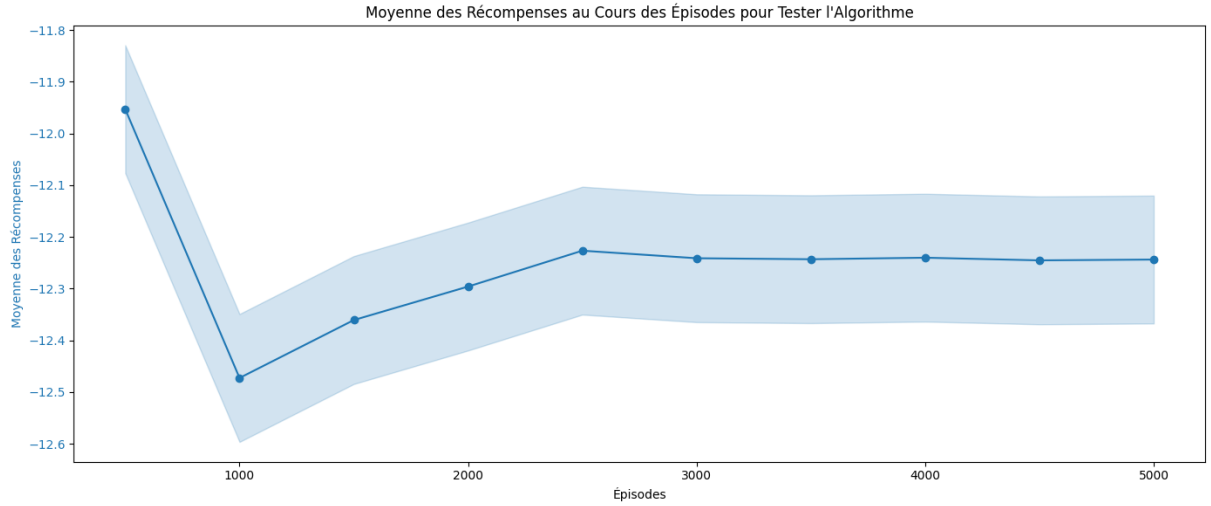


FIGURE 12 – Récompense moyenne pour le test de l'algorithme avec 7 véhicules.

Métrique	Mean	Std	Min	25%	50%	Max
Nombre d'étapes	19.905800	0.980571	6.0	20.0	20.0	20.0
Récompense moyenne	-12.252313	0.130327	-12.472692	-12.283345	-12.243494	-11.953153

TABLE 4 – Statistiques descriptives pour le scénario avec 7 véhicules.

La variation des commandes dans ce scénario au cours des 5 premiers épisode :

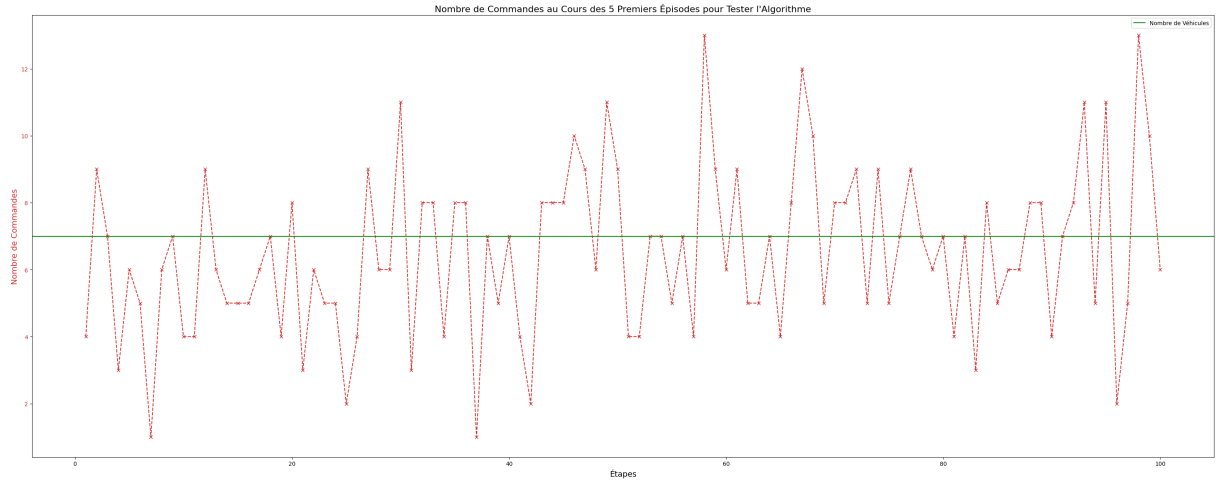


FIGURE 13 – Nombre de commandes au fil des étapes pour 7 véhicules.

5.3 Comparaison avec les politiques alternatives

Les résultats de l'algorithme ont été comparés à deux politiques alternatives : une prise de décision aléatoire et une maximisation des termes positifs (prix et urgence) de l'équation (1). Les Figures 14, 15, 16, 17, 18 et 19 montrent les récompenses moyennes obtenues pour chaque politique.

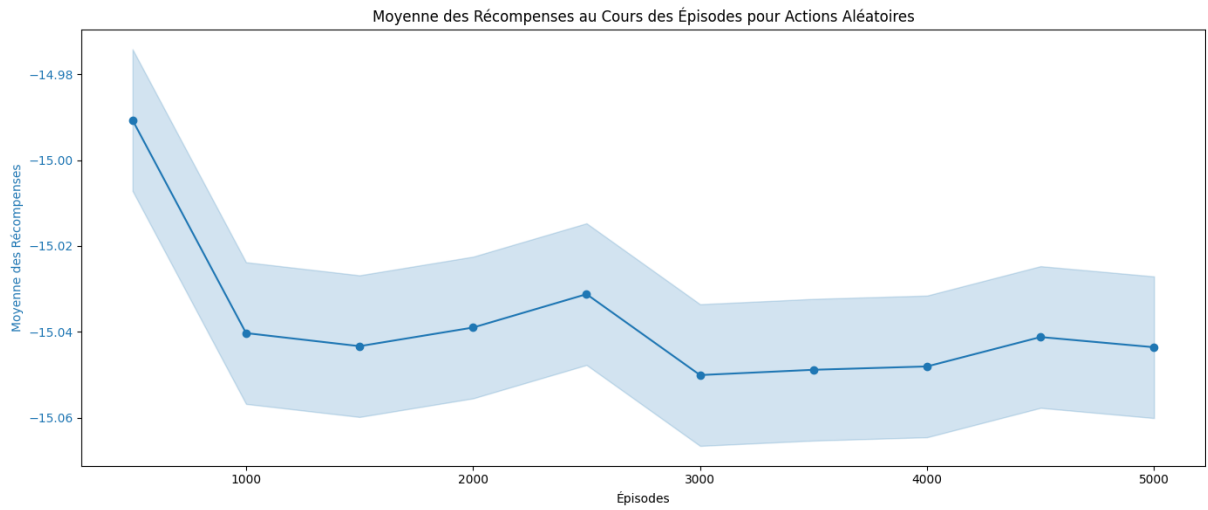


FIGURE 14 – Récompenses moyennes obtenues pour des actions aléatoires avec 3 véhicules.

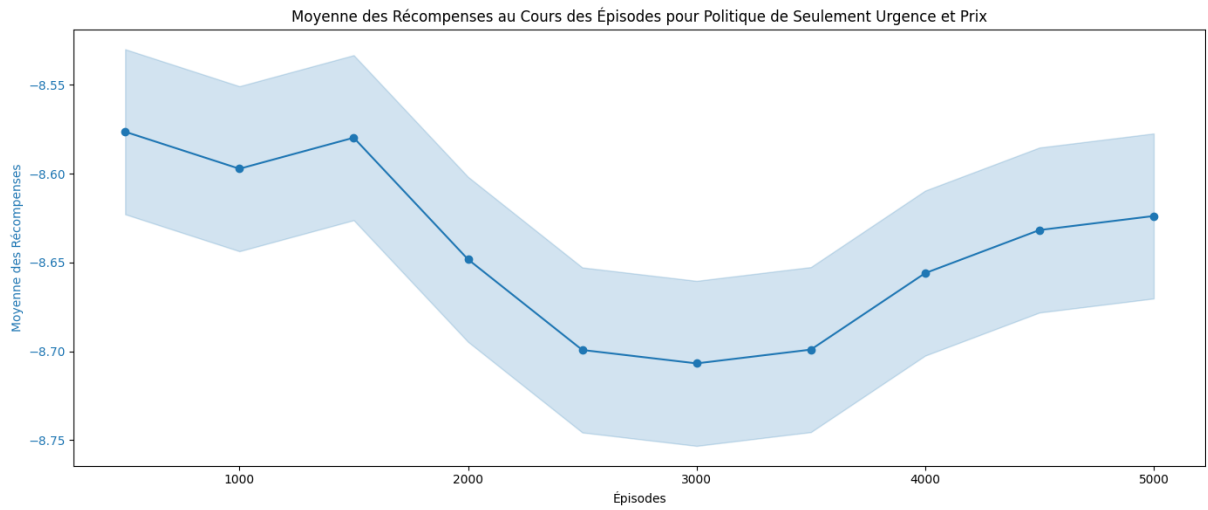


FIGURE 15 – Récompenses moyennes obtenues en valorisant uniquement le prix et l'urgence avec 3 véhicules.

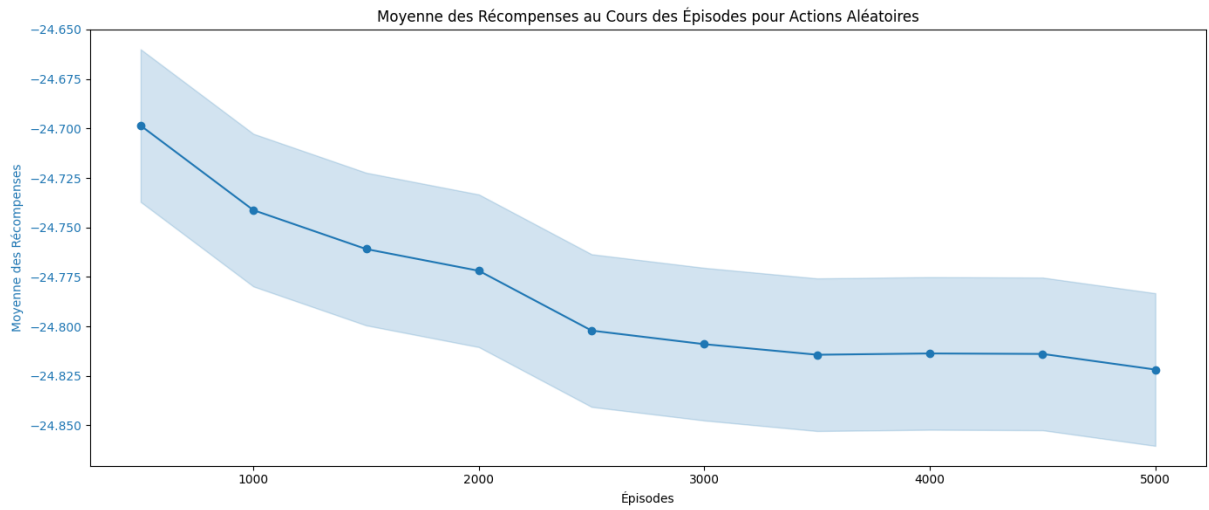


FIGURE 16 – Récompenses moyennes obtenues pour des actions aléatoires avec 5 véhicules.

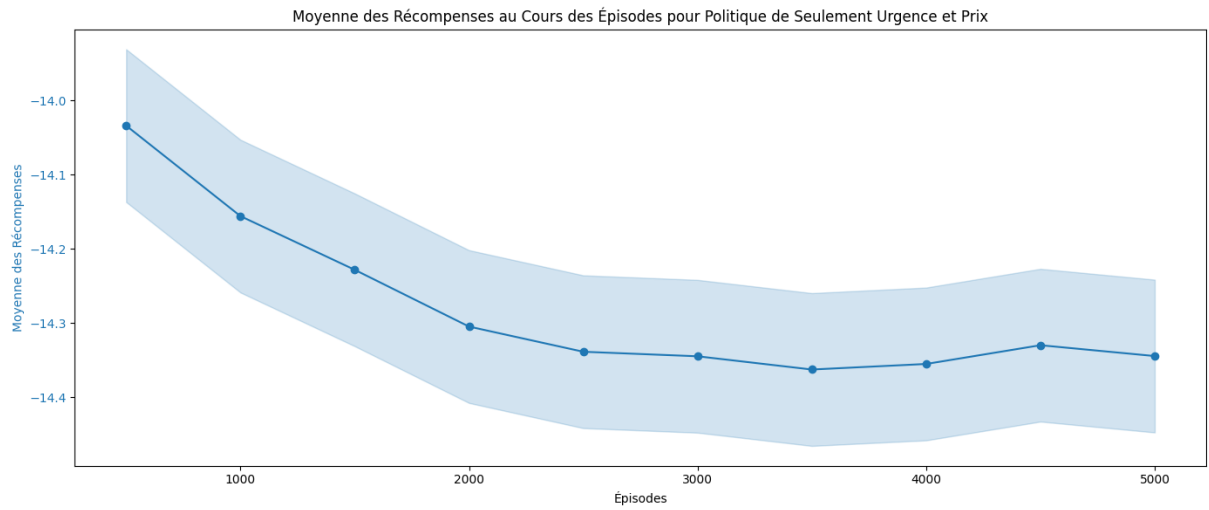


FIGURE 17 – Récompenses moyennes obtenues en valorisant uniquement le prix et l'urgence avec 5 véhicules.

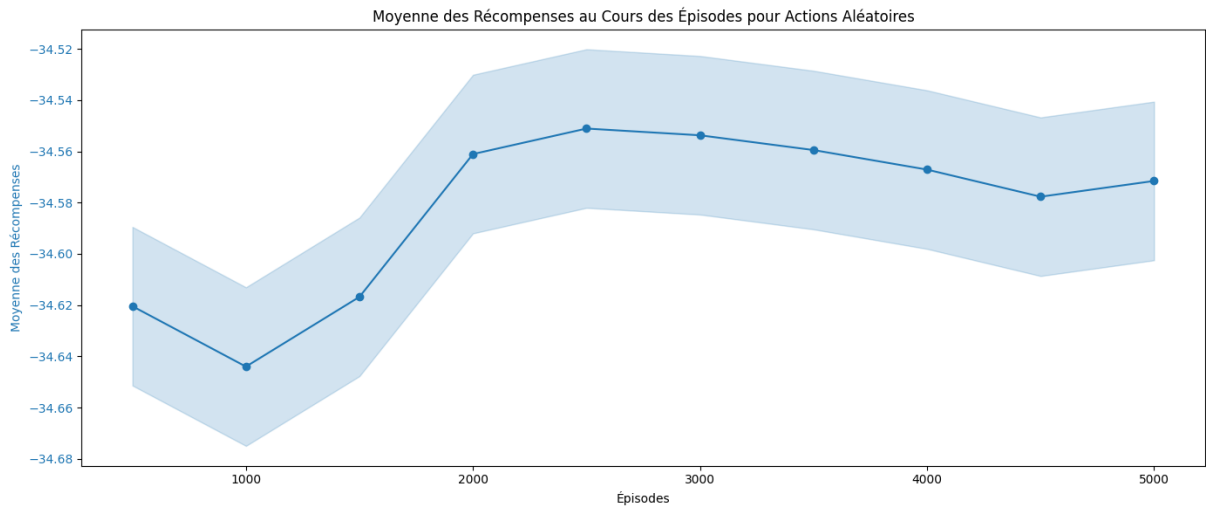


FIGURE 18 – Récompenses moyennes obtenues pour des actions aléatoires avec 7 véhicules.

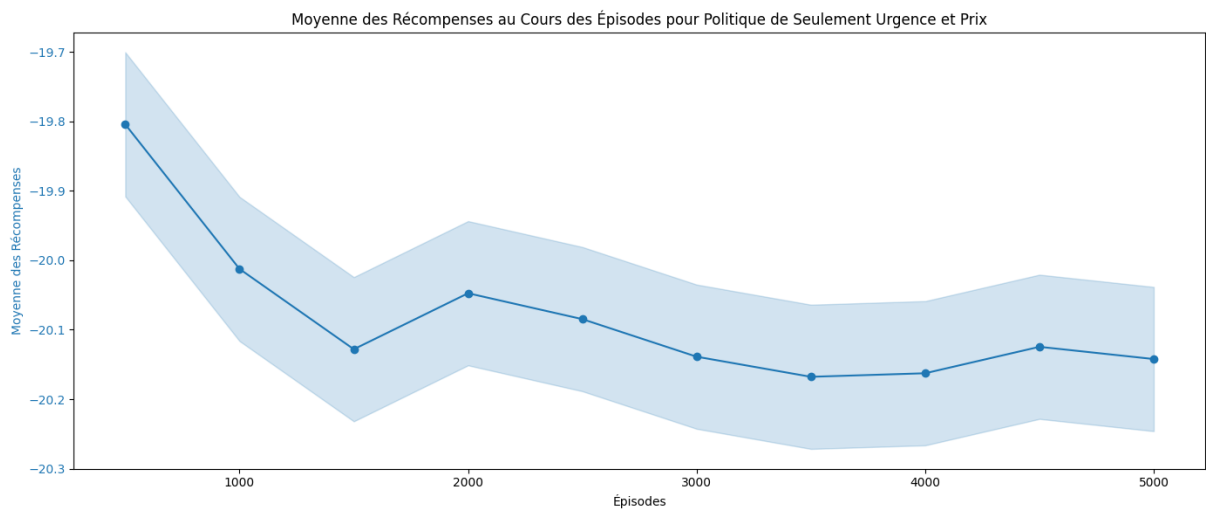


FIGURE 19 – Récompenses moyennes obtenues en valorisant uniquement le prix et l'urgence avec 7 véhicules.

5.4 Analyse de l'évolutivité

Le temps d'entraînement pour chacun des trois scénarios a également été calculé pour mesurer l'évolutivité de l'environnement et de l'algorithme appliqué. Le tableau ci-dessous présente les temps obtenus :

Nombre d'agents	Temps d'entraînement (minutes)
3	73
5	117
7	156

TABLE 5 – Temps d'entraînement pour différents nombres d'agents.

On observe ainsi une augmentation significative du temps d'entraînement à mesure que

le nombre d’agents augmente. Cependant, comme cette situation ne nécessiterait pas un entraînement constant, ce temps peut être considéré comme raisonnable compte tenu des résultats possibles obtenus grâce à cette application.

5.5 Discussion des résultats

Les résultats montrent que l’algorithme d’apprentissage par renforcement proposé est efficace pour maximiser les récompenses tout en évitant les états terminaux. Les comparaisons avec les politiques alternatives confirment sa supériorité en termes de performances et d’évolutivité. Cependant, certaines limitations doivent être notées. Par exemple, l’augmentation du temps d’entraînement avec le nombre d’agents pourrait poser des défis pour des environnements plus complexes. Des travaux futurs pourraient explorer des techniques d’optimisation du code pour réduire ce temps.

5.5.1 Comparaison des performances par scénario

Les tableaux suivants présentent une comparaison détaillée des performances entre l’algorithme proposé et les politiques alternatives pour chaque scénario, en incluant le nombre maximal d’étapes.

Métrique	Test	Actions aléatoires	Politique de prix et urgence
Récompense moyenne	3.706961	-15.037641	-8.641854
Nombre maximal d’étapes	20	15	14

TABLE 6 – Comparaison des performances pour le scénario avec 3 véhicules.

Métrique	Test	Actions aléatoires	Politique de prix et urgence
Récompense moyenne	-2.985293	-24.784739	-14.279917
Nombre maximal d’étapes	20	13	13

TABLE 7 – Comparaison des performances pour le scénario avec 5 véhicules.

Métrique	Test	Actions aléatoires	Politique de prix et urgence
Récompense moyenne	-12.252313	-34.582298	-20.081293
Nombre maximal d’étapes	20	13	12

TABLE 8 – Comparaison des performances pour le scénario avec 7 véhicules.

En résumé, les résultats montrent que l’algorithme d’apprentissage par renforcement proposé est efficace pour maximiser les récompenses tout en évitant les états terminaux. Les comparaisons avec les politiques alternatives confirment sa supériorité en termes de performances et d’évolutivité.

6 Conclusion

En conclusion, à partir du travail de modélisation réalisé et de l'application de l'apprentissage par renforcement, il est possible d'affirmer que l'utilisation de techniques d'apprentissage par renforcement pour le problème de recharge d'hydrogène s'avère prometteuse et produit des résultats très intéressants dans une dynamique de prise de décision pour orienter la prestation des services demandés. Bien que le temps d'entraînement ait tendance à augmenter avec la complexité de l'environnement et le nombre d'agents, la possibilité de ne pas avoir besoin de réaliser des entraînements multiples de manière constante rend cette approche avantageuse. Une fois entraîné, l'algorithme apporte des bénéfices significatifs, tels que l'amélioration de la qualité du service offert et un meilleur retour sur l'utilisation des véhicules, ce qui, à long terme, génère des avantages considérables.

Le code pour l'utilisation de l'environnement et des agents entraînés est disponible en libre accès dans [7].

Références

- [1] De Medeiros, C. L. L., "Outil d'Optimisation et Apprentissage par renforcement multi-agent pour le problème de planification en ligne d'un service mobile à la demande," *Rapport de stage de deuxième année d'ingénierie*, 2024.
- [2] Isidoro, M. H., "Outil d'optimisation et d'apprentissage multi-agent pour le problème de l'attribution de commandes et de véhicules à l'aide d'un algorithme basé sur le mariage stable," *Rapport de stage de deuxième année d'ingénierie*, 2024.
- [3] Teo, C.-P., Sethuraman, J., et Tan, W.-P., "Gale-Shapley Stable Marriage Problem Revisited : Strategic Issues and Applications", 2001.
- [4] Gale, D., et L. S. Shapley, "College Admissions and the Stability of Marriage", *The American Mathematical Monthly*, vol. 69, no. 1, 1962, pp. 9–15. <https://doi.org/10.2307/2312726>.
- [5] Sutton, R. S., et Barto, A. G., "Reinforcement Learning : An Introduction", 2018.
- [6] Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., et Mordatch, I., "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments", 2017.
- [7] De Medeiros, C. L. L., "Multi-Agent Hydrogen Recharge", *Dépôt GitHub*, disponible sur : <https://github.com/celiolucaslm/Multi-Agent-Hydrogen-Recharge.git>.